P3067R0
Named permutation functions for `std::simd`

Published Proposal, 2024-05-22

This version:
http://wg21.link/P3067R0

Authors:
Daniel Towner (Intel)
Ruslan Arutyunyan (Intel)

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

The paper proposes to provide a set of named functions for common permutation operations.

Table of Contents

§ 1. Motivation

ISO/IEC 19570:2018 (1) introduced data-parallel types to the C++ Extensions for Parallelism TS. [P1928R8] is proposing to make that a part of C++ IS. Intel supports the concept of a standard interface to SIMD instruction capabilities and in document [P2638R0] we made a number of suggestions for extending the capabilities of `std::simd` to handle named permutation for common operations like `insert`, `extract` and `resize`. Intel then also proposed in [P2664R6] an even more capable generic API for handling permutes using generated static indexes, dynamic indexes, selection by mask, and memory-based gather/scatter operations. In this paper we will revisit the original proposals from [P2638R0] and see how they can be modified in the light of the generic API, and from feedback received from the community.

§ 2. Background

While `std::simd` is undoubtedly most useful for expressing parallel operations (e.g., arithmetic) across entire sets of input arguments, many real-world algorithms require the data to be marshalled into new positions before operating on it. For example, modifying the order of values, concatenating or splitting data, or pulling out specific elements for some purpose. In the discussions around `std::simd` there have been of number of attempts to defined some sort of permute function, and these will be summarised below.

Our aim in this paper is to accept that the generic permute API discussed in [P2664R6] does everything that we could possible want in a permute API, but that it is useful to standardise a few specific and common types of permute so that users of `std::simd` aren't required to keep reinventing them.

In all of the summaries of other proposals below we can learn from the them and provide something that mimics the original intent with the new generic API.

§ 2.1. Original `std::simd` proposal

In [P1928R0] there were only a few functions which could be used to achieve any sort of element permutation across or within `std::simd<>` values. Those functions were relatively modest in scope and they built upon the ability to split and concatenate simd values at compile-time. For example, a simd value containing 5 elements could be split into two smaller pieces of sizes 2 and 3 respectively:

```
simd<float, 5> x;
…
const auto [piece0, piece1] = simd_split<3>(x);
```

Those smaller pieces could be acted on using any of the `std::simd` features, and then later recombined using `simd_cat`, possibly in a different order or with duplication:

```
const auto output = simd_cat(piece1, piece1, piece0);
```

The `simd_split`, `simd_split_by` and `simd_cat` functions can be used to achieve several permutation effects, but they are unwieldy for anything complicated, and since they are also compile-time they preclude any dynamic runtime permutation.

Since the original proposal a few changes have taken place. The original `split` function shown above, which could accept an arbitrary set of indexes to perform the split, was removed and the original `split_by` function was renamed to `split`. The new variant `split` now breaks a `std::simd` value into a set of identically sized pieces, and a remainder. The concatenation function remains in its original state.

As a small aside, the `split` operation is equivalent to what is called chunking in the views/ranges library, and `ranges::split` does something different. It may be better to call simd's function `simd_chunk` instead to avoid confusion.

One of the purposes of the original `split` and `concat` functions was to allow a large `std::simd` to be broken into pieces which had a native register size so that builtin operations (intrinsics) could be called on individual pieces, and the results glued back together. Such manual permutation for these purposes has been superseded by [P2929R0] which proposes a function called `simd_invoke` which automates this task.

§ 2.2. Suggestion to include selected named permute functions

Two suggestions were made in [P0918R1]: add an arbitrary compile-time shuffle function, and add a specific type of permutation called an interleave. The arbitrary shuffle has been replaced by a generated-permute function for reasons explained in [P2664R6] and will not be considered further here.

The other proposal was to add a function called interleave which accepts two SIMD values and interleaves respective elements into a new SIMD value which is twice the size. For example, given inputs `[a0 a1 a2]` and `[b0 b1 b2]`, the output would be `[a0 b0 a1 b1 a2 b2]`. Interleaving is a common operation and is often directly supported by processors with explicit instructions (e.g., Intel's `punpcklwd`), so as in [P0918R1] and also in other SIMD libraries there will be named functions which expose that instruction. This is a relatively common operations in certain workloads (e.g., signal processing) and so it was called out as a specific function to allow programmers from having to keep defining their own versions of what is a basic utility.

§ 2.3. `insert`, `extract` and `resize`

In [P2638R0] Intel suggested three new functions:

- `insert` - Place a copy of the elements from one `std::simd` within another `std::simd`, replacing the original elements at the given insertion position.
- `extract` - Create a smaller `std::simd` value from a sub-set of the elements of another `std::simd` value.
- `resize` - Grow or shrink a `std::simd` to a new number of elements.

These all operate purely statically, using fixed sizes and positions. There was a discussion of whether they should work dynamically too (e.g., insert data into some run-time specified position of another `std::simd`) but there was no consensus to pursue this.

There was feedback that the names should have more thought. For example, `insert` doesn't make the `std::simd` value into which the insertion is taking place any larger, but instead replaces elements. Also there was some concern that `resize` might silently do something unexpected, and that having functions which explicitly truncate (and lose) elements to shrink a `std::simd`, or explicitly grow a `std::simd` by adding value-initialised elements to reach the new size might be clearer. However, against this is that other containers in C++ (e.g., vector) have `resize` methods which ambiguously (and dynamically too) grow or shrink a container without warning.

In Intel's experience of using these functions in some of our software products we have found that no-one is using `insert`, but `extract` and `resize` are both found in real code.

Those functions can be implemented using the functions proposed in [P2664R6]. For example:

```
template<std::size_t _NewSize, typename _Tp, typename _Abi>
constexpr auto resize(const basic_simd<_Tp, _Abi>& v)
{
  constexpr int _OldSize = basic_simd<_Tp, _Abi>::size;
  return permute<_NewSize>(v,
              [](std::size_t i) { return (i < _OldSize) ? i : simd_zero_ele
}

template<std::size_t _Begin, std::size_t _End, typename _Tp, typename _Abi
constexpr auto extract(const simd_impl<_Tp, _OldSize>& v)
{
  return permute<_End - _Begin>(v, [](std::size_t i) { return i + _Begin;
}
```

§ 2.4. Named permute functions in other data parallel libraries

Other data parallel libraries provide functions for common permutation operations though often it will be to expose selected native instructions. For example, [Highway] provides `DupOdd`, `DupEven`, `ReverseBlocks`, `Shuffle0231`, and so on, which map efficiently to Intel instructions. In this case the choice of which functions to expose will guide the user to use permutes which are known to be efficient on a particular target, but this may come at the expense of portability.

§ 2.5. Powerful generic `std::simd` API

In [P2664R6] Intel proposed a comprehensive permutation API which covered the most common requirements of most users:

- Static permute generator - using a callable generator (e.g., lambda) to generate a static permutation. This can allow the compiler to perform arbitrary permutes in what are often very sophisticated and efficient ways.
- Dynamic permute using another `simd` value as an index.
- Mask-based permute where elements corresponding to the active elements of a `simd` are compressed or expanded, and inactive elements discarded.

- Memory-based permute where values are gathered from or scattered to a specific memory region (i.e., safely, with no access permitted outside the specificed region).

The subject of naming came into the discussions and at time of writing this hasn't been completely resolved (e.g., should there be a `simd_` prefix or a namespace?). Fundamentally there is nothing wrong with the set of permute functions or their behaviours.

One problem with the generic API though is that although it can do anything required, there are inevitably certain operations which are so common that every user of `std::simd` will write their own version of it. We received feedback that we should explore the provision of a set of common functions of this type, which brings us to the purpose of this paper.

§ 3. Addition of common utility functions

In our experience, the most useful of all the generic permute functions proposed in [P2664R6] is the generated permute. Here is an example to refresh the reader:

```
const auto dupEvenIndex = [](auto idx) { return (idx / 2) * 2; };
const auto de = permute(values, dupEvenIndex);
```

For every index position in the output value the generator lambda function is called with that index to determine what the source index should be for that element. In this particular example the generator is duplicating every even value.

The generator function can take both an index parameter and also a size parameter. This gives the ability to write a reversing permute generator:

```
auto rev = permute(s, [](auto idx, auto size) { return (size - 1) - idx; }
```

Finally, the output size of the generated simd may also be specified:

```
auto repeat_simd(simd<float, 3> s) {
  return permute<7>(s, [](auto idx, auto size) { return idx % size; });
}
```

In this example a simd value such as `[a, b, c]` will be repeated to fill 7 elements, forming the value `[a, b, c, a, b, c, a]`.

If the onus is placed on the user of `std::simd` to provide the generator functions for all types of permutes then they will inevitably duplicate common permute generators at any level from a single translation unit, in multiple files across a whole project, and even across unrelated projects. Within a file or project the user can obviously create a single permute generator which can be reused, but we don't want entirely different projects to have to keep duplicating common functions either.

Another issue with requiring the user to provide their own generators is that it is very easy to write something quite obscure. For example, `permute(x, [](auto i) { return i ^ 2; })` does not immediately tell the reader what is happening, but `permute(s, swap_adjacent)` is much clearer. With discipline the user can make the code clear with appropriate naming, but it adds to the workload of the user to get this right, and it would be simpler to use a standardised function provided by `std::simd` itself.

One final issue is that inevitably bugs will creep in to generator functions, and providing a set of known-working standard permute generators will improve development time by reducing debug time.

An initial step (and indeed, one that was suggested at the C++ committee meeting) is to propose a set of generator functions that can be dropped in to a generated permute. This feels initially promising, such as creating a lambda for reverse:

```
namespace simd_perm
{
  constexpr auto reverse = [](auto idx, auto size) { return (size - 1) - id
}


auto rev = permute(values, simd_perm::reverse);
```

Note that we have assumed that the function is put into some suitable namespace. We shall return to the question of naming these functions shortly.

Unfortunately following the idea of a named permute generator as above quickly leads to a problem; permutation often (even mostly) requires a change in size as well as a change in index order. For example, suppose we build a generator which reads values which are multiples of some constant - a

strided read. The generator can be defined and used as below. Note that we aren't using a lambda for this example because we wish to be able to template parameterise the stride distance.

```
template<int N>
struct stride {
    constexpr auto operator()(auto idx) const { return idx * N; };
};

auto everyThird = permute<3>(s, stride<3>{});;
```

In this example we have to provide how many output values we require, as well as the generator. If we don't provide the correct number then the permutation will fail as it tries to read invalid indexes. Therefore, for generic common permutation it isn't sufficient to provide named permute generators, we may also need to provide wrappers to make them useful.

To complete this particular example we would want a `simd` permute function for `stride` which was implemented as something like this:

```
namespace __details {
  template<int N>
  struct stride_helper {
    constexpr auto operator()(auto idx) { return idx * N; };
  };
}

namespace simd_permute {
  template<int N, typename _Simd>
  constexpr auto stride(const _Simd& s) {
    return permute<_Simd::size / N>(s, __details::stride_helper<N>{});
  }
}

auto everyThird = simd_permute::stride<3>(s);
```

We are faced with some small dilemma with how this relates to existing functions in the C++ libraries. There is already a range adaptor object `constexpr auto views::stride(Range&& r, DifferenceType&& n)` which has the same behaviour (i.e., pick out every N'th value of some input), but which works on a dynamic range. Our `simd`-specific version of stride differs in having to be called with a `simd` of a static size, return a `simd` of a different static size, and provide a template parameter rather than a variable parameter. This does change the function's signature and call-site use.

However, the advantage of reusing an existing name is that it is immediately clear what the intent and behaviour are supposed to be. We are not trying to invent a new name for an existing operation as that would require the user to learn an alternate name for every operations which already exists.

If we used the same name, but placed it inside a suitable namespace, then we will have to rely on either namespace qualifiers (e.g., simd_permutes::stride) to identify the correct function to call, or overloading:

```
using namespace simd_permutes;

auto r0 = stride<2>(s); // Call the stride operation on a simd.

auto r1 = stride(c, 2); // Call ranges::stride. This might even convert a s
```

For now we will assume that the `simd` version of such a function will be named to match the existing C++ name, but use overloading to differentiate it. Note that we could make the function signature more similar to what exists already if we used a `std::integral_constant` as a parameter, instead of using a template parameter, but this leads to verbose call sites. We discuss this option further later in this proposal, but for now use a template parameter to give a concise call site.

In the existing proposal we have a mixture of naming conventions. For example:

```
permute
compress
expand
simd_select
simd_cat
simd_split
```

Clearly we need to agree on a common naming convention. Most recently [P1928R8] is using `simd_` prefix on functions, but it may be more appropriate to provide namespaces. This topic will be discussed further in a separate paper. Until that is resolved, this proposal's main intent is to discuss the ideas of creating common functions, with their naming to be decided later.

§ 3.1. Named permute functions overview

Next we can consider what functions should be provided as part of the standard `simd` permute package. We should provide functions which are sufficiently common

and obvious that they will get wide-spread use. However, we should also avoid the temptation of providing functions which map onto specific native hardware implementations, unless the functions are truly useful on all targets. It isn't appropriate to expose functions which force a degree of target-specifity in a portable library. The list of suggested functions below takes as its inspiration suggestions from [P2664R6], [P2638R0], existing names from `std::ranges::views`, and the names of generator functions we use in Intel's software products. Note that all of these functions will apply to either `simd` or `simd_mask`, since they map onto the underlying `permute` functions which also work on either of those types.

Function	Behaviour
`take<int>(simd)`	Return a new `simd` containing the first N values. Replaces `resize` where the output must be smaller (i.e., a shrink). Can trap incorrect resizing (e.g., to something bigger).
`grow<int>(simd, value={})`	Return a new `simd` which is bigger than the original value by new elements with the given value, which defaults to value initialised. Replaces `resize` where the new size is bigger than the original.
`stride<int N>(simd)`	Return a new `simd` which contains every N'th element of the input.
`chunk<int>(simd)`	Existing `simd::split` renamed/moved.
`reverse(simd)`	Return a new `simd` of the same size with its elements in reverse order.
`repeat_all<int>(simd)`	Create a new `simd` which is N copies of the original (e.g., `repeat_all<3>([a b])` is `[a b a b a b]`)
`repeat_each<int>(simd)`	Create a new `simd` in which every element repeats N times (e.g., `repeat_each<3>([a b])` is `[a a a b b b]`)
`transpose<ROWS,COLS>(simd)`	Treat the `simd` as a row-major matrix with the given number of rows and columns and returns its transpose (i.e., in column major)
`zip(simd...)`	(aka interleave?) Generate a new `simd` which interleaves or zips corresponding element together. `zip([a b c], [0 1 2])` returns `[a 0 b 1 c 2]`. Useful for working with matrix-like values (e.g., signal processing).
`unzip<N>(simd...)`	(aka deinterleave?) Generate a new `simd` which deinterleaves the input. `unzip<2>([a 0 b 1 c 2])` gives `make_tuple([a b`

Function	Behaviour
	`c], [0 1 2])`. Useful for working with matrix-like values (e.g., signal processing).
`cat(simd...)`	Renamed version of `simd_cat`. Should it be called `join`? Should it take a tuple of simd values instead?
`extract<N, M>(simd)`	Extract a `simd` which contains the elements in the range [N..M). Equivalent to `drop<N> | take<M-N>`.
`rotate<MIDDLE>(simd)`	Perform a left rotation on the elements of the `simd` such that the element at the MIDDLE index becomes the first element. This matches the behaviour of `std::ranges::rotate` but not `std::rotl`/`std:rotr` which also allows negative rotations or rotations larger than the container (i.e., mod container size)
`shift_left<N>(simd)`/`shift_right<N>(simd)`	return a `simd` of the same size as the input but with the elements shifted left or right and value-initialised elements inserted. These would behave like `std::range::shift_left/right`. The disadvantage of these is that shifting a mask using this function would be the opposite of what might be expected. A `simd_mask` is similar to a plain set of bits, and shifting a set of bits left would move them towards the MSB. In contrast, applying `shift_left` to a `simd_mask` would move the bits towards the LSB. Maybe this shouldn't matter, because a `simd_mask` is not an integer which can be shifted, but conceptually it might be confusing. Also, should value initialised elements be shifted in, or should the new values be left valid but undefined?
`align<N>(simd a, simd b)`	Given two `simd` objects perform an extraction of a `simd` from `cat(a, b)`. Used to implement unaligned operations on targets where aligned operations are either inefficient or non-existant.

§ 3.2. Design option - functions or function objects

Although the paper has described a collection of "functions" to perform named permutes, we could make these all function objects which then gives us some more freedom in how they can be used. For example, in Intel's example implementation of `std::simd` we make them function objects which de-

rive from a named permute base-class, which then allows us to compose them into pipelines of permute transformations:

```
auto demo1(const simd<float>& x) {
    return x | rotate<5> | reverse;
}

auto demo2(const simd<float, 16>& x) {
    return x | transpose<4>;
}

auto even(const simd<float>& x) {
    return x | stride<2>;
}

auto odd(const simd<float>& x) {
    return x | drop<1> | stride<2>;
}

auto dupEven(const simd<float>& x) {
    return x | stride<2> | repeat_each<2>;
}

auto dupOdd(const simd<float>& x) {
    return x | drop<1> | stride<2> | repeat_each<2>;
}
```

We are not currently proposing that such pipelines should be formalised into the proposal, but we do ask for direction in deciding whether the suggested named permute functions should be functions or function objects.

§ 3.3. Design option - constant parameters

There is a discrepancy between the signatures of function like `views::ranges::take(e, N)` and `simd_permute::take<N>(s)`. Most `simd` permute functions operate on statically sized `simd` values with known element. It is therefore not possible to permute with dynamic sizing. Consequently the permute functions take constant template parameters, rather than dynamic parameters as with the `views::ranges` functions.

In order to more closely match the function signature from functions in `views::ranges`, we could modify the signature of `simd_permute::take` to be more similar:

```
template<typename _T, typename _Abi, int _N>
constexpr auto take(const basic_simd<_T, _Abi>& s,
                    std::integral_constant<int, _N>);


auto first3 = take(s, std::integral_constant<int, 3>{});
```

This gives a more familiar call site, but is rather verbose for the common case.

§ 4. Implementation Experience

One of the drivers for this paper was the observation that in software which used Intel's example implementation of `std::simd`, different developers were inventing their own versions of common permutes, often in slightly different ways. We wanted to guide different users to use a common language in the sorts of permutes that they should be using, and to avoid unnecessary duplication of effort in building what are sometimes quite sophisticated features from the underlying permute functions. We do still encounter permute functions which are unique in their behaviour, and don't deserve to be pulled out into a common library, and we have disregarded those functions in this proposal.

§ 5. Conclusion

In this paper we have set out an argument for providing a set of common permutation functions which have wide-spread use across different types of software which will use `std::simd`. There are still some open questions about naming conventions, but our main goal with this paper is to set out a proposed direction for what sorts of functions should be commonly provide, and to get feedback from the wider community.

§ References

§ Informative References

[Highway]
Google. *Google Highway*. URL: https://github.com/google/highway/

[P0918R1]

Tim Shen. *More simd<> Operations*. 29 March 2018. URL: https://wg21.link/p0918r1

[P1928R0]

Matthias Kretz. *Merge data-parallel types from the Parallelism TS 2*. 7 October 2019. URL: https://wg21.link/p1928r0

[P1928R8]

Matthias Kretz. *std::simd - Merge data-parallel types from the Parallelism TS 2*. 9 November 2023. URL: https://wg21.link/p1928r8

[P2638R0]

Daniel Towner. *Intel's response to P1915R0 for std::simd parallelism in TS 2*. 22 September 2022. URL: https://wg21.link/p2638r0

[P2664R6]

Daniel Towner, Ruslan Arutyunyan. *Proposal to extend std::simd with permutation API*. 16 January 2024. URL: https://wg21.link/p2664r6

[P2929R0]

Daniel Towner, Ruslan Arutyunyan. *simd_invoke*. 19 July 2023. URL: https://wg21.link/p2929r0